

Intel Arc Pro B70 + Qwen3.8-27B

vLLM XPU Community Replication Guide

A reproducible, community-oriented procedure for running Qwen3.8-27B GPTQ INT4 with BF16 MTP4 speculative decoding on a single Intel Arc Pro B70 32 GB GPU.

Validated baseline: 84.65 tokens/s median post-first-token decode at p512/g128, C1, using five measured runs after a same-shape warmup. This guide focuses only on the inference stack and general deployment characteristics.

1. What this guide reproduces

Item	Known-good value
GPU	Intel Arc Pro B70 32 GB
Model	SergiiOB/Qwen3.8-27B-GPTQ-Int4-sym-G128-MTP-BF16
Engine	vLLM XPU 0.27.2rc1.dev77+gac7509e2b.xpu
XPU kernels	vllm-xpu-kernels 0.1.12.3
Quantization	GPTQ INT4, symmetric, group size 128
Speculative decoding	MTP4 with BF16 draft tensors
KV cache	FP8
Context	131072
Benchmark	512 input tokens / 128 output tokens / concurrency 1
Measured median	84.65 tok/s
Measured median TTFT	0.3557 s

The goal is not to claim every B70 system will produce exactly 84.65 tok/s. CPU, PCIe topology, driver/kernel, power limits, thermals, Docker configuration, and software revisions can change results. Instead, this is a known-good reference that other B70 owners can use to validate their own systems.

2. Why this stack is interesting

- Qwen3.8-27B fits on one 32 GB B70 when using the documented GPTQ INT4 model and FP8 KV cache.
- MTP speculative decoding materially increases serial decode throughput compared with conventional decoding.
- The OpenAI-compatible vLLM server makes the result usable by many existing clients and applications.
- The 131072-token configured context gives substantial room for long prompts, document evidence, and retrieval-augmented generation.
- The configuration is especially attractive for local/private inference where a 27B-class model and high decode speed are desired on one GPU.

3. RAG-friendly characteristics

This configuration is well suited to retrieval-augmented generation (RAG). RAG systems typically retrieve a small set of relevant passages or records, place that evidence into the prompt, and ask the model to answer from the supplied context. Several properties of this B70/vLLM setup are helpful:

- Large context window: the 131072-token configuration provides ample headroom for retrieved passages, instructions, conversation history, and citations.
- Fast prefill capability: vLLM/XPU can process substantial prompt context efficiently, which matters because RAG adds retrieved evidence before generation.
- Fast serial decode: the reproduced ~84.65 tok/s p512/g128 result keeps answer generation responsive after the prompt has been processed.
- OpenAI-compatible API: common RAG frameworks and custom applications can call `/v1/chat/completions` or `/v1/completions` without a proprietary client protocol.

- Local execution: retrieved documents and prompts can remain on the local inference host when the rest of the RAG pipeline is also self-hosted.
- Long-context flexibility: the same server can support compact evidence prompts for low latency or much larger retrieved contexts when a task requires them.

For high-quality RAG, the model should not replace retrieval logic. A robust design retrieves authoritative evidence first, keeps the evidence concise, instructs the model to stay grounded in that evidence, and measures retrieval latency separately from model TTFT and decode speed.

4. Host prerequisites

- Linux host with a working Intel xe driver for the Arc Pro B70.
- Docker installed and able to pass /dev/dri devices into containers.
- A B70 with 32 GB VRAM visible through a render node such as /dev/dri/renderD128 or /dev/dri/renderD129.
- Healthy PCIe link. Check for accidental x1 or low-generation operation before judging inference performance.
- Adequate GPU cooling and power delivery. Avoid benchmarking while the GPU is thermally or power throttled.
- Enough local disk space for the approximately 19 GB model plus Docker images and caches.

Device numbering is system-specific. Do not assume the B70 is card2/renderD129. Identify the correct Intel GPU and render node on your host.

5. Identify the B70 and DRI nodes

```
lspci -nn | grep -Ei 'VGA|Display|Intel'

ls -l /dev/dri
ls -l /dev/dri/by-path 2>/dev/null || true

for d in /sys/class/drm/card*/device; do
    echo "=== $d ==="
    readlink -f "$d"
    cat "$d/vendor" 2>/dev/null
    cat "$d/device" 2>/dev/null
done
```

Record the B70 PCI address, card node, render node, and render-group GID. Substitute your render node wherever this guide uses renderD129.

6. Verify PCIe health

```
# Replace 04:00.0 with the B70 PCI address found above.
sudo lspci -vv -s 04:00.0 | grep -E 'LnkCap|LnkSta|LnkCap2|LnkCtl2'
```

A severely downgraded link can distort results. The validation system used for this guide was operating at PCIe Gen3 x16. A different platform may negotiate another healthy width/generation appropriate to its hardware.

7. Cookbook, image, model, and patches

Repository:
<https://github.com/SergioB/intel-arc-pro-b70-inference-cookbook>

Local example:
 ~/intel-arc-pro-b70-inference-cookbook

Pinned image:
 vllm/vllm-openai-xpu@sha256:f01e24f6c7ff01f1e0662234255a1372297d1dbd89d003cf13c8fad3eab1ba4f

Model:
 SergioB/Qwen3.8-27B-GPTQ-Int4-sym-G128-MTP-BF16

```

Model revision:
9d189a60e4c0ad7f9f47cd94bfa393ca10b3924e

Example local model directory:
$HOME/models/Qwen3.8-27B-GPTQ-Int4-sym-G128-MTP-BF16

Required patches:
patches/patch_mtp_nightly.py
SHA256:
4d7a02c4ea10ca7c00dc89ad927fa3dafa747dbf0553d2adf24e30a3c53e9c14

patches/patch_mtp_boundary.py
SHA256:
41d2f74e5fef1f074b76b5a90dd1016de437228431802cfb1fa7bd7ce4cc9b50

```

Pinning the image, model revision, and patch hashes is important for reproducibility. Upstream cookbook and vLLM behavior can change over time.

8. Verify the patch files

```

cd ~/intel-arc-pro-b70-inference-cookbook

sha256sum patches/patch_mtp_nightly.py
sha256sum patches/patch_mtp_boundary.py

```

9. Critical Docker / oneCCL requirement

A common failure point is XCCL/oneCCL initialization inside Docker. Passing only an individual render node may not be sufficient. The validated workaround exposes the complete DRI directory and supplies the IPC/security permissions below.

```

--device /dev/dri:/dev/dri
-v /dev/dri:/dev/dri:ro
--group-add "$RENDER_GID"
--cap-add SYS_PTRACE
--security-opt seccomp=unconfined
--ipc=host

```

Without these flags, the validation setup failed in oneCCL's `ze_fd_manager` path with an inability to open the device directory. With them, XCCL initialized successfully.

10. Known-good launch command

Before running this command, replace `renderD129` if your B70 uses a different render node. Change host port 11436 if that port is already occupied.

```

cd ~/intel-arc-pro-b70-inference-cookbook

export IMAGE='vllm/vllm-openai-xpu@sha256:f01e24f6c7ff01f1e0662234255a1372297d1dbd89d003cf13c8fad3eab1ba4f'
export MODEL_DIR="$HOME/models/Qwen3.8-27B-GPTQ-Int4-sym-G128-MTP-BF16"
export B70_RENDER_NODE="/dev/dri/renderD129"
export RENDER_GID="$(stat -c '%g' "$B70_RENDER_NODE")"

docker rm -f qw38speed 2>/dev/null || true

docker run -d \
  --name qw38speed \
  --restart unless-stopped \
  -p 11436:8000 \
  --device /dev/dri:/dev/dri \
  -v /dev/dri:/dev/dri:ro \
  --group-add "$RENDER_GID" \
  --cap-add SYS_PTRACE \
  --security-opt seccomp=unconfined \
  --ipc=host \
  -v "$MODEL_DIR:/model:ro" \
  -v "$PWD/patches/patch_mtp_nightly.py:/patch_mtp.py:ro" \

```

```

-v "$PWD/patches/patch_mtp_boundary.py:/patch_boundary.py:ro" \
-e VLLM_TARGET_DEVICE=xpu \
-e ZE_FLAT_DEVICE_HIERARCHY=COMPOSITE \
-e ZE_AFFINITY_MASK=0 \
-e B70_MTP_BF16_DRAFT=1 \
-e VLLM_XPU_ENABLE_XPU_GRAPH=1 \
-e PYTORCH_ALLOC_CONF=expandable_segments:True \
--entrypoint bash \
"$IMAGE" \
-lc 'set -e
python /patch_mtp.py
python /patch_boundary.py
exec vllm serve /model \
--quantization gptq \
--dtype float16 \
--max-model-len 131072 \
--gpu-memory-utilization 0.88 \
--kv-cache-dtype fp8 \
--port 8000 \
--max-num-seqs 64 \
--max-num-batched-tokens 8192 \
--no-enable-prefix-caching \
--served-model-name qwen38 \
--language-model-only \
--speculative-config "{\"method\":\"mtp\",\"num_speculative_tokens\":4}"'

```

11. Multi-GPU hosts: selecting the intended B70

The validated host contained more than one Intel GPU. `ZE_AFFINITY_MASK=0` selected the B70 on that machine, but affinity numbering can differ. On a different host, verify which Level Zero device index corresponds to the B70 before assuming index 0. The strongest sanity check after launch is that vLLM reports memory consistent with a 32 GB B70 (approximately 30.3 GiB usable/detected in the validated setup).

12. Startup and health verification

```

docker logs -f qw38speed

# In another terminal after startup:
curl -fsS http://127.0.0.1:11436/health && echo "HEALTH OK"

curl -fsS http://127.0.0.1:11436/v1/models | jq

```

Do not benchmark until the server log reports: Application startup complete.

- Confirm model id qwen38 appears in `/v1/models`.
- Confirm no oneCCL/XCCL fatal error appears.
- Confirm the model is loaded with the AutoGPTQ XPU path.
- Confirm MTP drafter initialization occurs.
- Confirm XPU graph capture completes.
- Confirm detected device memory is appropriate for the B70.

13. First inference: warmup is mandatory

The first request can trigger Triton JIT compilation for EAGLE/MTP kernels. This produces a large one-time latency penalty. Always warm up the same prompt/output shape before recording benchmark numbers.

```

Possible first-run warnings:
Triton kernel JIT compilation during inference:
  eagle_prepare_next_token_padded_kernel
  eagle_step_slot_mapping_metadata_kernel
  eagle_prepare_inputs_padded_kernel

```

These warnings are not necessarily a failure. If they occur only during the cold request and disappear on the repeated shape, benchmark the warmed runs.

14. Verify MTP speculative decoding

After inference, inspect the server logs for SpecDecoding metrics. A working MTP path should report drafted and accepted tokens, acceptance length, and per-position acceptance rates. In the validated run, early examples showed mean acceptance length around 2.7 and average draft acceptance around 42–43%.

```
docker logs qw38speed 2>&1 | tail -40
```

15. Correct benchmark methodology

Two measurements can be misleading for short requests: vLLM's periodic throughput logger and raw SSE chunk counts. The reliable method used here is client-side streaming timing combined with the API's final completion_tokens usage count.

- Do not interpret 'Avg generation throughput' from the periodic vLLM logger as the p512/g128 card result for a short isolated request.
- Do not assume one SSE event equals one token. Multiple generated tokens can be coalesced into a single streaming chunk.
- Use one same-shape warmup, then five measured concurrency-1 requests.
- Use monotonic client timing.
- Compute post-first-token decode as $(\text{completion_tokens} - 1) / (\text{request_end} - \text{first_generated_token})$.

16. Reproduction benchmark: p512/g128

```
python3 - <<'PY'
import json, time, urllib.request, statistics

BASE = "http://127.0.0.1:11436"

seed = (
    "A retrieval augmented generation system retrieves relevant source passages, "
    "constructs a grounded context, sends the evidence to a language model, and "
    "generates a concise answer based on the supplied information. "
)

text = seed
while True:
    req = urllib.request.Request(
        BASE + "/tokenize",
        data=json.dumps({"model": "qwen38", "prompt": text}).encode(),
        headers={"Content-Type": "application/json"},
    )
    with urllib.request.urlopen(req) as r:
        tok = json.load(r)
        if len(tok["tokens"]) >= 512:
            break
        text += seed

ids = tok["tokens"][:512]

req = urllib.request.Request(
    BASE + "/detokenize",
    data=json.dumps({"model": "qwen38", "tokens": ids}).encode(),
    headers={"Content-Type": "application/json"},
)
with urllib.request.urlopen(req) as r:
    text = json.load(r)["prompt"]

def run():
    payload = {
        "model": "qwen38",
        "prompt": text,
        "max_tokens": 128,
        "temperature": 0,
        "ignore_eos": True,
        "stream": True,
```

```

        "stream_options": {"include_usage": True}
    }

    req = urllib.request.Request(
        BASE + "/v1/completions",
        data=json.dumps(payload).encode(),
        headers={"Content-Type": "application/json"},
    )

    start = time.perf_counter()
    first = None
    end = None
    usage = None

    with urllib.request.urlopen(req, timeout=300) as r:
        for raw in r:
            line = raw.decode().strip()
            if not line.startswith("data: "):
                continue

            data = line[6:]
            if data == "[DONE]":
                end = time.perf_counter()
                break

            obj = json.loads(data)

            if obj.get("usage"):
                usage = obj["usage"]

            choices = obj.get("choices", [])
            if choices and choices[0].get("text") and first is None:
                first = time.perf_counter()

    if end is None:
        end = time.perf_counter()

    ct = usage["completion_tokens"]
    pt = usage["prompt_tokens"]

    return {
        "prompt": pt,
        "completion": ct,
        "ttft": first - start,
        "post": end - first,
        "rate": (ct - 1) / (end - first)
    }

print("Warmup...")
w = run()
print(f"warmup: {w['rate']:.2f} tok/s  TTFT={w['ttft']:.4f}s")

rates = []
ttfts = []

for i in range(1, 6):
    r = run()
    rates.append(r["rate"])
    ttfts.append(r["ttft"])
    print(
        f"run {i}: {r['rate']:.2f} tok/s  "
        f"TTFT={r['ttft']:.4f}s  "
        f"prompt={r['prompt']} completion={r['completion']}"
    )

print()
print(f"MEDIAN DECODE: {statistics.median(rates):.2f} tok/s")
print(f"MEAN DECODE: {statistics.mean(rates):.2f} tok/s")
print(f"MIN/MAX: {min(rates):.2f} / {max(rates):.2f} tok/s")
print(f"MEDIAN TTFT: {statistics.median(ttfts):.4f} s")
PY

```

17. Validated reference result

Warmup:
83.82 tok/s

Measured runs:
83.73 tok/s
84.65 tok/s
84.70 tok/s
84.61 tok/s
84.78 tok/s

MEDIAN DECODE: 84.65 tok/s
MEAN DECODE: 84.49 tok/s
MIN/MAX: 83.73 / 84.78 tok/s
MEDIAN TTFT: 0.3557 s

The spread was only about 1.05 tok/s from minimum to maximum. A result in the same general range indicates that the stack is behaving similarly. Do not treat small differences as failures; platform and software differences matter.

18. Performance context

Inference path	Qwen3.8-27B format	Observed decode
Ollama Vulkan	Q4_K_M	~11–12 tok/s on validation host
llama.cpp SYCL	Q4_K_M	~27.4 tok/s on validation host
vLLM XPU + MTP4	GPTQ INT4 + BF16 draft	84.65 tok/s median

These comparisons came from one validation system and should not be generalized as universal engine ratios. They demonstrate why the XPU/MTP path was worth pursuing on the B70.

19. Power and thermal checks

```
# Replace card2 if the B70 is another DRM card.
for h in /sys/class/drm/card2/device/hwmon/hwmon*; do
    echo "=== $h ==="
    cat "$h/name" 2>/dev/null
    cat "$h/power1_cap" 2>/dev/null
    cat "$h/power1_cap_max" 2>/dev/null
done
```

Inspect rather than blindly writing a power limit. The reference campaign used a 230 W configured cap. Also verify temperatures, clocks, and throttling during sustained tests.

20. Common failure modes

Item	Known-good value
oneCCL/XCCL ze_fd_manager error	Expose full /dev/dri, bind it, add render GID, SYS_PTRACE, seccomp=unconfined, and host IPC.
Wrong Intel GPU selected	Verify Level Zero affinity and confirm memory size corresponds to the 32 GB B70.
Very slow first request	Look for Triton JIT warnings; repeat the same shape after warmup.
~12.8 tok/s in vLLM log	Periodic logger artifact for short isolated requests; use client-side streaming timing.
Very low SSE 'token' count	SSE chunks are not tokens; use completion_tokens from stream usage.
MTP seems inactive	Inspect SpecDecoding drafted/accepted-token metrics.
Unexpectedly poor throughput	Check PCIe link width/speed, power, clocks, thermals, and software pins.
Port conflict	Change the host side of -p 11436:8000; container port remains 8000.

21. Reproducibility checklist

- Record Linux distribution and kernel.
- Record Intel xe driver state and B70 PCI address.
- Record PCIe negotiated speed and width.
- Record B70 card/render nodes and Level Zero affinity selection.
- Record Docker version.
- Pin the vLLM image by digest, not only by tag.
- Pin the model revision.
- Verify patch SHA256 hashes.
- Record vLLM command-line arguments and environment variables.
- Record GPU power cap, clocks, temperatures, and throttling state.
- Warm the exact benchmark shape before measuring.
- Report prompt tokens, generated tokens, concurrency, number of repetitions, median, and timing definition.

22. Safe baseline before optimization

Once this baseline works, preserve it before testing newer optimization overlays. Optional draft-INT4, MTP-INT4, GDN mixed-split, or newer cookbook patches can produce different performance and memory behavior. Treat each as a separate configuration with its own image digest, patch hashes, launch command, and benchmark results.

23. Quick recovery sequence

1. Identify the B70 PCI address and render node.
2. Verify the PCIe link is healthy.
3. Clone/open the cookbook repository.
4. Confirm the pinned model is present.
5. Verify the two required patch hashes.
6. Set B70_RENDER_NODE to the correct render node.
7. Launch the container with the full /dev/dri + oneCCL Docker flags.
8. Wait for "Application startup complete."
9. Verify /health and /v1/models.
10. Run one same-shape warmup.
11. Run five p512/g128 measured requests.
12. Compare the median with the ~84 tok/s known-good reference.
13. If results differ substantially, inspect PCIe, device selection, JIT warmup, MTP metrics, power/thermals, and software revisions before changing tuning knobs.

24. Reference URLs

Cookbook:

<https://github.com/SergioB/intel-arc-pro-b70-inference-cookbook>

Qwen3.8-27B vLLM XPU guide:

<https://github.com/SergioB/intel-arc-pro-b70-inference-cookbook/blob/master/docs/qwen38-27/QWEN38-VLLM-XPU.md>

Full setup commands:

<https://github.com/SergioB/intel-arc-pro-b70-inference-cookbook/blob/master/docs/FULL-SETUP-COMMANDS.md>

Document scope: community replication of a known-good single-B70 Qwen3.8-27B vLLM XPU/MTP configuration. No private application architecture, proprietary data, or business-specific deployment details are included.